MongoDB Database Architecture & Pipeline Guide

Comprehensive Technical Specification for Smart Attendance & Web Application Systems

Backend Specification | MongoDB BSON Types | Authentication Pipeline

1. Overview of MongoDB & BSON Data Types

MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like documents called **BSON** (Binary JSON). Unlike traditional relational SQL tables with rigid schemas, MongoDB documents allow nested objects, arrays, and native binary types.

BSON / Mongoose Type	Internal Representation	Usage & Application in Backend
`ObjectId`	12-byte unique BSON binary	Primary key (`_id`) and foreign key references (e.g. `student: ObjectId` pointing to `User`).
`String`	UTF-8 text string	Used for textual attributes such as names, emails, roll numbers, hashed passwords, and categories.
`Number` / `Int32` / `Double`	64-bit IEEE 754 float / 32-bit int	Used for numerical values, roll number bounds, quantities, and statistics.
`Date`	64-bit integer (milliseconds since Epoch)	Used for event tracking (`createdAt` , `date` , expiry timestamps).
`Boolean`	1-byte boolean (true / false)	Flags such as `isLab` , active status, or verified flags.
`Array`	List of values / sub-documents	Grouped sets, such as student lab batches `batches: [{ batchName, startRoll, endRoll }]` .

2. Smart Attendance System — Database Collections

The system database schema consists of **5 core collections** defined in Mongoose. Below is the complete field specification:

2.1 Users Collection (`models/User.js`)

Stores authentication records and demographic details for administrators, teachers, and students.

Field Name	Type	Constraints / Indexes	Description & Purpose
`_id`	`ObjectId`	Primary Key (Auto)	Unique identifier generated automatically by MongoDB.
`name`	`String`	Required, Trimmed	Full legal name of the user.
`email`	`String`	Unique, Sparse, Lowercase	Email address used for Teacher and Admin authentication.
`password`	`String`	Required, `select: false`	Hashed using `bcrypt` with 10 salt rounds. Hidden from default queries for security.
`role`	`String`	Enum: `['admin', 'teacher', 'student']`	Determines platform permissions and routing. Default is `'student'`. Indexed for fast lookup.
`rollNumber`	`String`	Unique, Sparse	College roll number used by Students for sign-in.
`subject`	`String`	Sparse	Primary instructional subject assigned to teachers.
`department`	`String`	Indexed Compound	Department code (e.g. `CSE`, `ECE`, `MECH`).
`semester`	`String`	Indexed Compound	Current semester term (e.g. `4th`, `6th`).
`createdAt`	`Date`	Default: `Date.now`	Timestamp when the account profile was provisioned.

2.2 Attendance Collection (`models/Attendance.js`)

Stores individual attendance log records with normalized timestamps and multi-key compound indexes.

Field Name	Type	Reference	Description & Purpose
`_id`	`ObjectId`	—	Unique attendance record identifier.
`student`	`ObjectId`	Ref: `User`	The student whose attendance is recorded.
`teacher`	`ObjectId`	Ref: `User`	The instructor who logged the record.
`subject`	`ObjectId`	Ref: `Subject`	Associated academic subject.
`date`	`Date`	—	Date of lecture normalized to midnight (`00:00:00`) for consistent date-range queries.
`status`	`String`	—	Attendance state: `'Present'` , `'Absent'` , or `'Late'` .
`section`	`String`	—	Class section or batch identifier (Default: `'Regular Class'`).
`createdAt`	`Date`	—	Audit timestamp when the entry was written to the database.

2.3 Subject Collection (`models/Subject.js`)

Contains course curricula, faculty assignments, and lab batch divisions.

Field	Type	Description
`name`	`String`	Subject name (e.g. *Database Management Systems*).
`code`	`String (Unique)`	Unique academic code (e.g. `CS401`).
`teacher`	`ObjectId (Ref: User)`	Assigned faculty member responsible for the curriculum.
`department` / `semester`	`String`	Target academic department and semester.
`isLab`	`Boolean`	Indicates whether this course involves laboratory sessions.
`batches`	`Array of Objects`	List of lab subdivisions: `{ batchName: String, startRoll: Number, endRoll: Number }` .

2.4 Notice Collection (`models/Notice.js`)

Central notice board for announcements and administrative circulars.

Field	Type	Description
`title`	`String`	Notice headline.
`message`	`String`	Detailed notice body.
`category`	`String`	Enum: `'General'` , `'Examination'` , `'Urgent'` , `'Academic'` , `'Other'` .
`postedBy`	`ObjectId (Ref: User)`	Author administrator or faculty member.
`targetDepartment` / `targetSemester`	`String`	Audience filters (empty string indicates broadcast to all students).
`createdAt` / `updatedAt`	`Date`	Timestamps managed automatically by Mongoose timestamps option.

2.5 Material Collection (`models/Material.js`)

Repository of study resources, lecture slides, and download links.

Field	Type	Description
`title`	`String`	Title or chapter description.
`subject` / `subjectCode`	`ObjectId` / `String`	Linked subject entity and quick lookup code.
`department` / `semester`	`String`	Target curriculum division.
`fileUrl` / `link`	`String`	File storage endpoint or web URL.
`originalFileName`	`String`	Original name of uploaded document.
`uploadedBy`	`ObjectId (Ref: User)`	Uploader ID for audit control.

3. End-to-End Data Flow Architecture (Login Pipeline)

When a user interacts with the application, data travels through multiple network and software layers before reaching MongoDB. Below is the complete step-by-step lifecycle:

1. **Client Form Submission:** The user selects their role (`Student` , `Teacher` , or `Admin`) and enters credentials. Students enter their **Roll Number** and **Password**; Teachers/Admins enter their **Email** and **Password**.
2. **HTTPS POST Request:** The browser executes an asynchronous `fetch()` or `axios.post()` call sending JSON over TLS to `/api/auth/login` .
3. **Express.js Route & Body Parser:** The Node.js server receives the packet, executes `express.json()` middleware, and populates `req.body` .
4. **MongoDB Query Execution:** The controller determines the query criteria and calls `User.findOne().select('+password')` over the MongoDB native driver connection pool.
5. **Bcrypt Password Validation:** The server executes `bcrypt.compare(enteredPassword, user.password)` to cryptographically verify the password against the stored salt hash.
6. **JWT Issuance:** Upon successful verification, the server signs a JSON Web Token containing `{ id, role }` with an expiry of 30 days and sends it back in the response body.

> **Security Architecture: Why Password Hashing is Mandatory**
>
> Passwords are never stored in plain text. When a user registers or updates their password, a Mongoose pre-save hook automatically executes: `const salt = await bcrypt.genSalt(10); this.password = await bcrypt.hash(this.password, salt);` Even if the database is exposed, the actual passwords cannot be reversed.

3.1 Controller Implementation in Your Codebase

```
// Location: backend/controllers/authController.js
const loginUser = async (req, res) => {
    try {
        const { email, rollNumber, password, role } = req.body;

        let user;
        // Role-based lookup logic
        if (role === 'student' && rollNumber) {
            user = await User.findOne({ rollNumber, role }).select('+password');
        } else if (email) {
            user = await User.findOne({ email, role }).select('+password');
        }

        // Cryptographic password comparison
        if (user && (await user.matchPassword(password))) {
            res.json({
                _id: user._id,
                name: user.name,
                email: user.email,
                role: user.role,
                rollNumber: user.rollNumber,
                token: generateToken(user._id, user.role)
            });
        } else {
            res.status(401).json({ message: 'Invalid credentials or unauthorized role' });
        }
    } catch (error) {
        res.status(500).json({ message: 'Login failed', error: error.message });
    }
};
```

> **Database Performance & Indexing Note**
>
> Your project implements compound indexes on the Attendance collection: `attendanceSchema.index({ student: 1, date: -1, subject: 1 });` This ensures that when a student views their semester history, MongoDB performs an index scan (IXSCAN) in O(log N) time rather than scanning every document in the collection.